

Configuration d'un réseau ad hoc avec un ensemble de raspberry-pi

Pour configurer le réseau j'ai utilisé **Network-manager** , un programme permettant de détecter et de configurer les systèmes pour qu'ils se connectent automatiquement aux réseaux .

Configuration des raspberry pi

Précision :

dhcpcd implémente un client DHCP en configurant des adresses IP et d'autres paramètres à l'aide du protocole DHCP qui est utilisé pour attribuer dynamiquement des adresses IP à chaque hôte du réseau de votre organisation. Mais celui ci peut entrer en conflit avec network manager ce pourquoi on doit le désactiver

Le **BSSID** est choisi au hasard juste on doit avoir comme condition que sur le 1^{er} octet le bit 1 soit activé, adresse **localement administrée** (non attribuée par un fabricant)

Bit 0 à 0 → adresse **unicast**

02 en hexadécimal = **00000010** en binaire → les deux conditions sont respectées. C'est pourquoi on commence par **02** : et les 6 octets du BSSID sont en hexadécimale

Protocole :

Effectuer les mises à jour : `sudo apt update && sudo apt upgrade`

Installer Network-Manager : `sudo apt install network-manager -y`

Désactivation dhcpcd (conflit avec NM) :

- `sudo systemctl stop dhcpcd`
- `sudo systemctl disable dhcpcd`

Activer Network-Manager :

- `sudo systemctl enable NetworkManager`
- `sudo systemctl start NetworkManager`

Vérifier le statut initial : `iwconfig wlan0`

Cette commande nous permet de voir si le mode ad hoc est activé dans le champ " MODE" qui vaut "managed" si ce n'est pas le cas . (normal pour première configuration)

si Mode="managed" on va devoir le changer en unmanaged , cela spécifie a network manager que l'on ne souhaite plus qu'il s'occupe du mode afin de pouvoir le définir manuellement.

Forcer IBSS (ad hoc) manuellement

```
sudo nmcli device set wlan0 managed no
```

```
sudo ip link set wlan0 down
```

 " ip link gère les interfaces réseau , set permet de cibler wlan0 qui doit être éteint avant de changer son type "

```
sudo iw dev wlan0 set type ibss
```

 " type ibss correspond à ad hoc "

```
sudo ip link set wlan0 up
```

```
sudo iw dev wlan0 ibss join mon-mesh-pi 2437 02:AA:BB:CC:DD:EE
```

 " créer notre réseau ad hoc "

```
sudo ip addr add 192.168.99.X/24 dev wlan0
```

 X = 1, 2, 3 ou 4 selon le Pi

Vérifications finales

```
iwconfig wlan0
```

 # doit afficher Mode:Ad-Hoc

```
ping 192.168.99.X
```

 # tester la communication entre Pi

Configuration d'un réseau ad hoc + Batman avec un ensemble de raspberry-pi

On doit installer premièrement Batctl qui est un outils de configuration et de débogage pour batman-adv

La plupart des configurations de batman-adv sont gérées par une famille de socket " batadv" qui sert de lien entre le noyau linux et l'interface user. batcl donne juste une interface commande pour cela.

pour les options voir

<https://www.open-mesh.org/projects/batman-adv/wiki/Using-batctl>

```
sudo apt install batctl -y
```

```
sudo apt install iw
```

Charger le module batman-adv depuis le kernel linux

```
sudo modprobe batman-adv
```

Supprimer l'IP de wlan0 (elle va migrer sur bat0)

```
sudo ip addr del 192.168.99.X/24 dev wlan0
```

ip : outils de gestion réseau , addr : sous commande pour gérer les ip

Ajouter wlan0 comme interface batman-adv

```
sudo batctl if add wlan0
```

if : outil réseau

Activer bat0

```
sudo ip link set up dev bat0
```

Assigner l'IP sur bat0

```
sudo ip addr add 192.168.99.X/24 dev bat0
```

Voir les interfaces actives dans batman

```
sudo batctl if
```

Voir les voisins détectés (doit afficher l'autre Pi)

```
sudo batctl n
```

Automatisation de la configuration d'un réseau ad hoc + protocole batman

On met l'ensemble des commandes que l'on avait tapé précédemment dans un fichier .sh

rendre le fichier exécutable

```
sudo chmod +x /usr/local/bin/batman-start.sh
```

Puis on crée un fichier service afin que celui-ci se lance au démarrage

Un fichier .service est divisé en 3 parties, la première **[unit]** est plus indicative, description c'est ce que l'utilisateur voit dans les fichiers et after : network.target démarre le service une fois que le réseau est activé, **[service]** dit comment exécuter le service, type oneshot : on exécute le code une seule fois, ExecStart définit le chemin vers le code à exécuter et remainAfterExit permet qu'une fois le script terminé le service est considéré comme actif par systemd, on peut vérifier son état avec "systemctl status" et **[install]** nous dit quand démarrer le service, ici quand plusieurs utilisateurs peuvent se connecter au raspberry pi.

